

Grading Instructions — CDMP (Canadian Donation Map)

These are the steps a TA/Professor needs to run and grade the system. The recommended path uses Docker with a pre-built Mongo dump, so no manual data pipeline needs to be run. Full background is in DOCKER.md and README.md in the repo root; this page is the condensed grading path.

1. Prerequisites

- Docker Desktop installed and running
- The data handoff zip from Google Drive (contains the MongoDB dump and raw data). The current package name is referenced in README.md under “Docker and Data Handoff”; check that section for the most recent Google Drive link, since the package is refreshed periodically.
 - <https://drive.google.com/file/d/1xQCDA2nmejBlbpYF7N54gwZZ8EDKn5G/view?usp=sharing>
- (Optional, for Ask CDMP) An API key from an AI provider; OpenAI, Anthropic Claude, or Google Gemini. Not required to grade the map or research features; see Section 3 below.

2. Clone and Start the App

1. Clone the repository and enter it:

```
git clone  
<https://github.com/UTSC-CSCC01-Software-Engineering-I/course-project-nacss.git>  
cd course-project-nacss
```

2. Download the current handoff zip from Google Drive into the project root and unzip it:

```
unzip -n course-project-nacss-drive-upload-<date>.zip
```

3. Confirm this file exists after unzipping:

```
mongo-dump/course_project_nacss.archive
```

4. Create the backend environment file (required, see Section 3 for why):

```
cp server/.env.example server/.env
```

5. Open server/.env and make the two required edits described in Section 3, then start the containers:

```
docker compose up --build -d
```

3. Configuring server/.env (Required Before Starting Containers)

Docker **does** read server/.env, the docker-compose.yml server service sets env_file: ./server/.env, so this file must exist before docker compose up is run, and its values do reach the container. (Compose separately overrides MONGODB_URI to point at the internal mongo service, so that one line can be left alone.)

Required: replace JWT_SECRET

server/.env.example ships with a placeholder:

JWT_SECRET=replace_with_a_long_random_value. This is a publicly known string (it's in the

repo), so it must be replaced with an actual random value before grading, login/session signing should not run on a secret every clone of the repo shares. Generate one and paste it in:

```
openssl rand -hex 32
```

Set the result as JWT_SECRET in server/.env. Any long random string works; it does not need to match anything else.

Optional: enable Ask CDMP with an AI provider key

The map and research dashboard work with no AI key at all. Ask CDMP specifically needs AI_API_KEY filled in, without it, the panel reports the AI service is unavailable rather than failing silently. Only one provider is active at a time, selected by AI_PROVIDER.

- **OpenAI** — create a key at platform.openai.com/api-keys (requires billing set up). Use:

```
AI_PROVIDER=openai-compatible
AI_API_KEY=<your key>
AI_MODEL=gpt-4o-mini
AI_BASE_URL=https://api.openai.com/v1
```

- **Anthropic Claude** — create a key at console.anthropic.com → Settings → API keys. Use:

```
AI_PROVIDER=anthropic
AI_API_KEY=<your key>
AI_MODEL=claude-sonnet-4-6
AI_BASE_URL=
```

- **Google Gemini** — create a key at aistudio.google.com/apikey (has a free tier). Use:

```
AI_PROVIDER=gemini
AI_API_KEY=<your key>
AI_MODEL=gemini-3.5-flash-lite
AI_BASE_URL=
```

Note: for Claude and Gemini, AI_BASE_URL must be left empty. If it still holds the OpenAI URL from the template, requests for those two providers will go to the wrong endpoint and fail.

Leave as-is: PASSWORD_RESET_PREVIEW

Security note: keep PASSWORD_RESET_PREVIEW=false for grading. When true, it skips emailing a reset link (since the project has no third-party email integration) and instead returns the reset link directly in the browser response, with no proof the requester owns that email address. That is a real account-takeover bypass, not a demo convenience; it should only be flipped to true briefly if a TA specifically wants to click through the reset flow live, then set back to false immediately after.

server/.env summary for grading

- MONGODB_URI — leave as-is; Docker overrides it.
- JWT_SECRET — required. Replace the placeholder with a real random value (see above).

- PASSWORD_RESET_PREVIEW — required to stay false for grading.
- AI_PROVIDER / AI_API_KEY / AI_MODEL / AI_BASE_URL — optional; only needed to grade Ask CDMP. Pick one provider from the table above.
- CLIENT_URL, HOST, PORT, REQUIRE_MONGODB — leave as the shipped defaults.

4. Restore the Database

```
docker compose run --rm mongo-tools mongorestore --host mongo --drop
--archive=/dump/course_project_nacss.archive
```

Confirm the restore:

```
docker compose --profile tools run --rm data-importer npm run check:data
```

Expected key counts:

```
Donation records: 5374039
Region stats: 8159
Postal-riding mappings: 3347996
Donation riding assignments: 5374039
```

5. Open and Verify the App

- Frontend: <http://localhost:8080>
- Backend health check: <http://localhost:5001/api/health>

Stop the app when done grading:

```
docker compose down
```

Do not run the “Full Data Rebuild Path” in DOCKER.md for grading; it is only needed to validate the raw data pipeline end to end and takes much longer. The Mongo dump restore above is the intended grading/demo path.

6. Testing Public Features

No login is required for:

- The national donation map, province drill-down, and riding-level maps
- Filters: party, year, donor type, and boundary-vintage bucket
- Search (national and riding views)
- Per-capita view toggle
- The riding summary panel (total donations, donor count, party breakdown, longitudinal trend)

7. Testing Research Features

There is no seeded researcher account in the Mongo dump, so create one through the app itself:

1. Go to **Research Login** and click **Sign Up**.
2. Enter any email ending in .ca or .edu (e.g. ta@utoronto.ca) and a password (8+ characters).
3. Log in with those credentials.
4. Accept the privacy agreement when prompted; this is required before any individual-level data is shown.
5. You now have access to the research dashboard, where you can:
 - Filter individual donation records by donor type, province, party, year, riding, and donor name search
 - Page through results
 - View the analytics panel (top ridings by donation count, donation amount/concentration distribution) for the current filter set
 - Export the current filtered result set as a CSV
 - View and update account info, including changing your password

All logins, queries, and exports are written to the activity log for audit purposes; there is no dedicated admin UI to view this log in the current release, but the underlying records live in the ActivityLog collection in MongoDB if you want to inspect them directly (e.g. via mongosh or mongo-tools).

8. Testing Ask CDMP (Optional)

Only testable if an AI provider key was configured in Section 3. If no key is set, skip this section — the panel will correctly show an “unavailable” message rather than an error.

- Open the map page; the “Ask CDMP” panel is in the side panel.
- Ask a supported question, e.g. “How much was donated nationally in 2023?” or “Which party received the most donations in 2023?”, and confirm a CDMP answer card is returned.
- Try a follow-up (e.g. “Compare Liberals and Conservatives instead.”) to confirm context is preserved.
- Try an out-of-scope question (e.g. asking for an individual donor’s name) to confirm it is refused rather than answered.

9. Running the Automated Test Suites

These do not require Docker or a database connection.

Backend (Jest + Supertest):

```
cd server
npm install
npm test
```

Frontend unit tests (Jest):

```
cd client
npm install
```

```
npm test
```

End-to-end tests require the app to be running via Docker on `http://localhost:8080` first:

```
docker compose up -d  
cd client  
npx cypress run
```

10. Troubleshooting

- **Restore fails with “archive not found”** — confirm the file is at `mongo-dump/course_project_nacss.archive`.
- **Map or dashboard APIs return empty data** — restore the Mongo dump again.
- **Ask CDMP says the AI service is unavailable** — recheck `AI_PROVIDER`, `AI_API_KEY`, `AI_MODEL`, and `AI_BASE_URL` in `server/.env` (see Section 3), then `docker compose up -d` to reload it.
- **Login/register fails unexpectedly** — confirm `JWT_SECRET` in `server/.env` is set to a real value, not the `replace_with_a_long_random_value` placeholder, then recreate the server container.
- **Docker build fails in client** — run `cd client && npm install` locally to refresh dependencies, then rebuild.
- **Need to reset all data** — `docker compose down -v`, then restore the dump again.

Full detail on every command above, plus the full pipeline rebuild path, is in `DOCKER.md`.